Prosperity Prognosticator: Machine Learning for Startup Success Prediction

Introduction

Project Title: Prosperity Prognosticator: Machine Learning for Startup Success Prediction
Team Details
Internship Program: SmartBridge Virtual Internship
Project Start Date: 29 January 2026
Team ID: LTVIP2026TMIDS80982
Team Size: 4
Team Leader: Parasa Lakshmi Sruthi
Team Members:

- Vundamatla Vishnu Manikanta
- Rankireddy Durga Bhima Shankar
- Vamsi Odugu

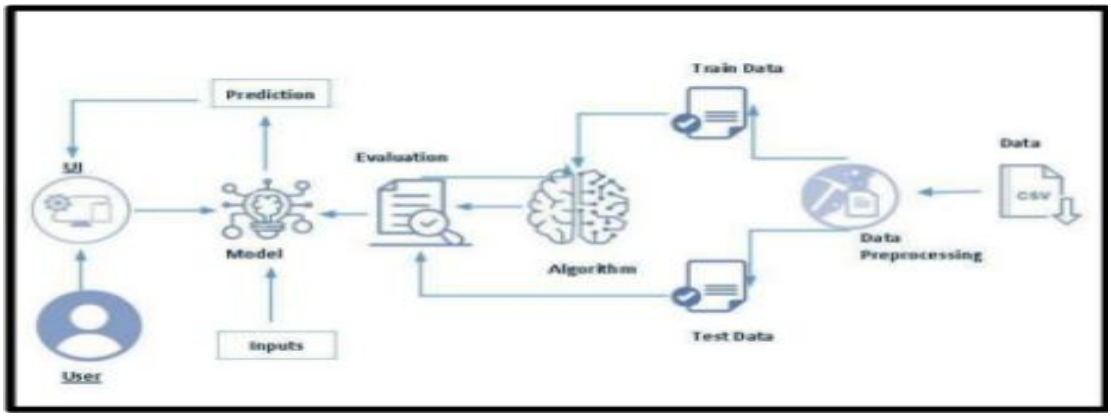
Project Description:

Startup ventures play a vital role in driving innovation, economic growth, and employment in modern society. The success of a startup mainly depends upon multiple factors such as market demand, funding availability, business model, competition, and management strategy. Due to the uncertainty and dynamic nature of business environments, accurately predicting startup success becomes a challenging task.

Predicting the success probability of startups in advance helps investors, entrepreneurs, and stakeholders make informed decisions, allocate resources effectively, and reduce financial risks. Machine Learning techniques provide efficient methods to analyze historical startup data and identify patterns associated with successful and failed ventures.

In this project, Machine Learning techniques are used to predict startup success based on key business and financial parameters. The dataset containing features such as funding details, industry sector, growth indicators, and performance metrics is analyzed and processed. A prediction model is trained and tested using training and testing datasets to obtain accurate results.

Technical Architecture:



Pre-Requisites:

To complete this project, the following software and packages are required:

Anaconda Navigator is an open-source software distribution used for package management and deployment of Python applications.

Python Packages:

- Open anaconda prompt as administrator
- Type “pip install numpy” and click enter.
- Type “pip install pandas” and click enter.
- Type “pip install scikit-learn” and click enter.
- Type ”pip install matplotlib” and click enter.

Prior Knowledge :

You must have prior knowledge of following topics to complete this project.

- **ML Concepts**
  - Supervised learning: https://www.javatpoint.com/supervised-machine-learning
  - Unsupervised learning: https://www.javatpoint.com/unsupervised-machine-learning
  - Regression and classification
  - Decision tree: https://www.javatpoint.com/machine-learning-decision-tree-classification-algorithm
  - Random forest: https://www.javatpoint.com/machine-learning-random-forest-algorithm
  - KNN: https://www.javatpoint.com/k-nearest-neighbor-algorithm-for-machine-learning
  - Xgboost: https://www.analyticsvidhya.com/blog/2018/09/an-end-to-end-guide-to-understand-the-math-behind-xgboost/
  - Evaluation metrics: https://www.analyticsvidhya.com/blog/2019/08/11- important-model-evaluation-error-metrics/
- **Flask Basics** : https://www.youtube.com/watch?v=lj4I_CvBnt0

Project Objectives:

By the end of this project you will:

- Understand Machine Learning prediction concepts
- Gain knowledge about startup business datasets
- Learn data preprocessing techniques for business and financial data
- Understand the relationship between startup features (funding, market, growth, etc.) and success outcomes
- Learn how to build and evaluate startup success prediction models

Project Flow:

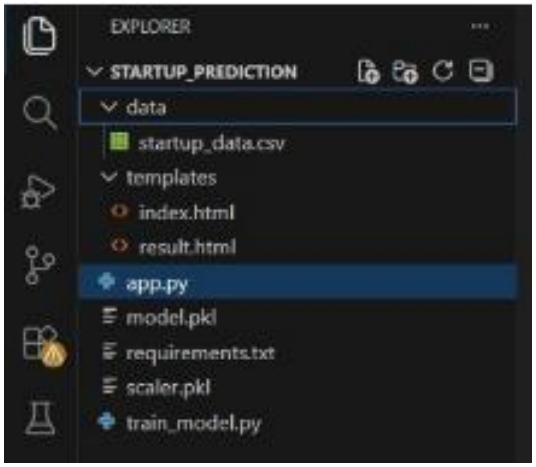
- User provides input dataset
- Dataset is analyzed
- Model is trained using historical wind data
- Model predicts the wind power generation

To accomplish this, we have to complete all the activities listed below,

- Data collection
- Collect the dataset or create the dataset
- Visualizing and analyzing data
- Univariate analysis
- Bivariate analysis
- Multivariate analysis
- Descriptive analysis
- Data pre-processing
- Checking for null values
- Handling outlier
- Handling categorical data
- Splitting data into train and test
- Model building
- Import the model building libraries
- Initializing the model
- Training and testing the model
- Evaluating performance of model
- Save the model
- Application Building
- Create an HTML file
- Build python code

Project Structure:

Create the Project folder which contains files as shown below



- The **templates** folder contains all the HTML files such as *index.html*, *result.html* which are used to build the user interface of the Startup Success Prediction web application.
- The **Training** folder includes the dataset file (*startup_data.csv*) and the Python script (*train_model.py*) which are used for data preprocessing, training and testing the Machine Learning model.
- The **model.pkl** file stores the trained Machine Learning model which is used by the application to predict startup output based on user input parameters.
- The **app.py** file acts as the backend of the Flask application which connects the user interface with the trained model and displays the prediction results.

Milestone 1: Data Collection

Machine Learning depends heavily on data. Dataset can be downloaded from open-source platforms or created manually.

Link:

https://www.kaggle.com/datasets/manishkc06/startup-success-prediction

Milestone 2: Visualizing and Analyzing the Data

Activity 1: Importing Libraries

Required libraries such as NumPy, Pandas and Matplotlib are imported for dataset analysis.

```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, roc_auc_score

import joblib
```

Activity 2: Reading the Dataset

Dataset is read using pandas from CSV, Excel or Text files.

```
data = pd.read_csv("/content/startup data.csv")
data.head(), data.shape
```

```
(   Unnamed: 0 state_code   latitude   longitude zip_code       id  \
 0        1005         CA  42.358880  -71.056820    92101   c:6669
 1         204         CA  37.238916 -121.973718    95032  c:16283
 2        1001         CA  32.901049 -117.192656    92121  c:65620
 3         738         CA  37.320309 -122.050040    95014  c:42668
 4        1002         CA  37.779281 -122.419236    94105  c:65806

            city              Unnamed: 6               name  labels  ...  \
 0      San Diego                     NaN        Bandsintown       1  ...
 1      Los Gatos                     NaN          TriCipher       1  ...
 2      San Diego      San Diego CA 92121              Plixi       1  ...
 3      Cupertino      Cupertino CA 95014  Solidcore Systems       1  ...
 4  San Francisco  San Francisco CA 94105     Inhale Digital       0  ...

   object_id has_VC has_angel has_roundA  has_roundB  has_roundC  has_roundD  \
 0    c:6669      0         1          0           0           0           0
 1   c:16283      1         0          0           1           1           1
 2   c:65620      0         0          1           0           0           0
 3   c:42668      0         0          0           1           1           1
 4   c:65806      1         1          0           0           0           0
```

Activity 3: Dataset Analysis

```
data.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 923 entries, 0 to 922
Data columns (total 49 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Unnamed: 0                923 non-null    int64
 1   state_code                923 non-null    object
 2   latitude                  923 non-null    float64
 3   longitude                 923 non-null    float64
 4   zip_code                  923 non-null    object
 5   id                        923 non-null    object
 6   city                      923 non-null    object
 7   Unnamed: 6                430 non-null    object
 8   name                      923 non-null    object
 9   labels                    923 non-null    int64
 10  founded_at                923 non-null    object
 11  closed_at                 335 non-null    object
 12  first_funding_at          923 non-null    object
 13  last_funding_at           923 non-null    object
 14  age_first_funding_year    923 non-null    float64
 15  age_last_funding_year     923 non-null    float64
 16  age_first_milestone_year  771 non-null    float64
 17  age_last_milestone_year   771 non-null    float64
 18  relationships             923 non-null    int64
```

The dataset is analyzed to understand:
Startup funding variation, industry sector distribution, and success outcomes.
The relationship between startup features (such as funding amount, market category, business age, and growth indicators) and startup success is identified using visualization techniques.

Milestone 3: Data Pre-processing

Collected dataset is cleaned before training the model.
Steps involved:
Handling Missing Values
Handling Categorical Data
Normalization
Feature Selection
Splitting Dataset

Activity 1: Checking for Null Values

Dataset shape and data types are checked using:
df.shape df.info() df.isnull().sum()

```
data.describe()
```

	Unnamed: 0	latitude	longitude	labels	age_first_funding_year	age_last_funding_year	age_first_milestone_year	age_last_milestone_year
count	923.000000	923.000000	923.000000	923.000000	923.000000	923.000000	771.000000	771.000000
mean	572.297941	38.517442	-103.539212	0.646804	2.235630	3.931456	3.055353	4.754423
std	333.585431	3.741497	22.394167	0.478222	2.510449	2.967910	2.977057	3.212107
min	1.000000	25.752358	-122.756956	0.000000	-9.046600	-9.046600	-14.169900	-7.005500
25%	283.500000	37.388869	-122.198732	0.000000	0.576700	1.669850	1.000000	2.411000
50%	577.000000	37.779281	-118.374037	1.000000	1.446600	3.528800	2.520500	4.476700
75%	866.500000	40.730646	-77.214731	1.000000	3.575350	5.560250	4.686300	6.753400
max	1153.000000	59.335232	18.057121	1.000000	21.895900	21.895900	24.684900	24.684900

8 rows x 35 columns

Activity 2: Handling Categorical Values

Categorical values are converted into numerical format for model training.

Handle Missing Values

```
df.fillna(df.mean(), inplace=True)
```

Generate + Code + Markdown

Activity 3: Splitting Dataset

Datasetis split into:

Independent Variables (X)

Dependent Variable (Y)

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)
```

Training and Testing datasets are created:

80% Training Data

20% Testing Data

Milestone 4: Model Building

After preprocessing, the dataset is trained using Machine Learning models.
Steps involved:

Import Model Libraries

Initialize Model

Train Model

Test Model

Evaluate Performance

Activity 1: Random forest model

A function named randomForest is created and train and test data are passed as the parameters. Inside the function, RandomForestClassifier algorithm is initialized and

training data is passed to the model with .fit() function. Test data is predicted with .predict() function and saved in new variable. For evaluating the model, confusion matrix and classification report is done.

```
scaler = StandardScaler()

model = RandomForestClassifier(
    n_estimators=200,
    random_state=42,
    class_weight='balanced'
)

# Fit scaler and model
X_train_scaled = scaler.fit_transform(X_train)
model.fit(X_train_scaled, y_train)
```

```
RandomForestClassifier
RandomForestClassifier(class_weight='balanced', n_estimators=200,
                       random_state=42)
```

Model Evaluation

Model performance is evaluated using:

Training Accuracy

Testing Accuracy

Model Evaluation

```
y_pred = model.predict(X_test)

r2 = r2_score(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))

print("R2 Score:", r2)
print("RMSE:", rmse)
```

```
X_test_scaled = scaler.transform(X_test)
y_pred = model.predict(X_test_scaled)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.7942238267148014
              precision    recall  f1-score   support

           0       0.81      0.55      0.65        98
           1       0.79      0.93      0.85       179

    accuracy                           0.79       277
   macro avg       0.80      0.74      0.75       277
weighted avg       0.80      0.79      0.78       277
```

The trained model is used to predict wind energy generation.

Milestone 5: Application Building

In this section, we will be building a web application that is integrated to the model we built. A UI is provided for the uses where he has to enter the values for predictions. The enter values are given to the saved model and prediction is showcased on the UI.
This section has the following tasks

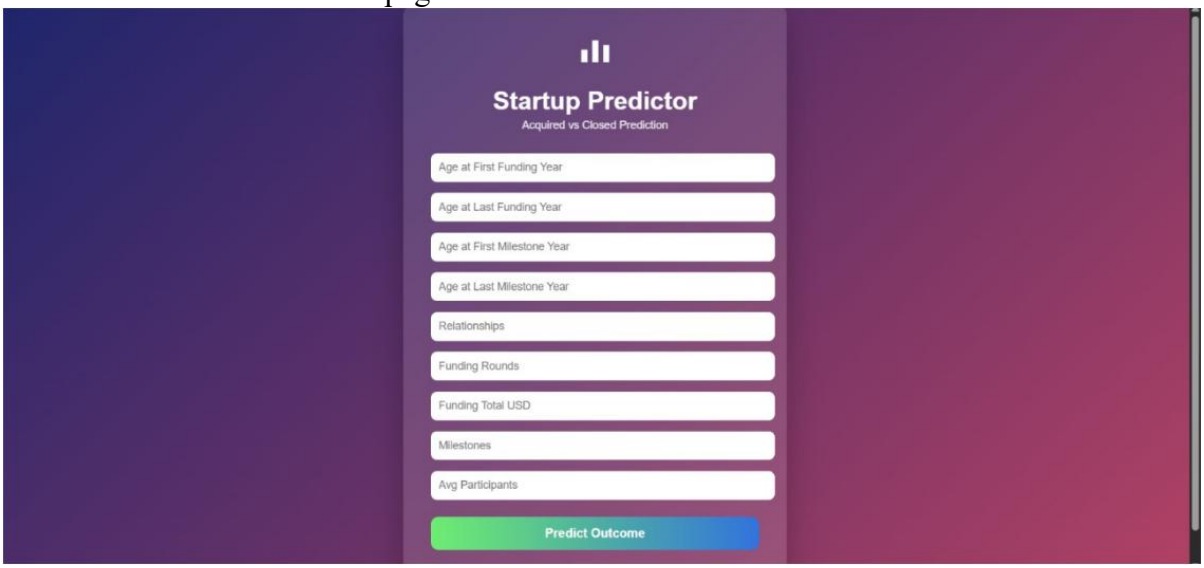
- Building HTML Pages
- Building serverside script

Activity1: Building Html Pages:

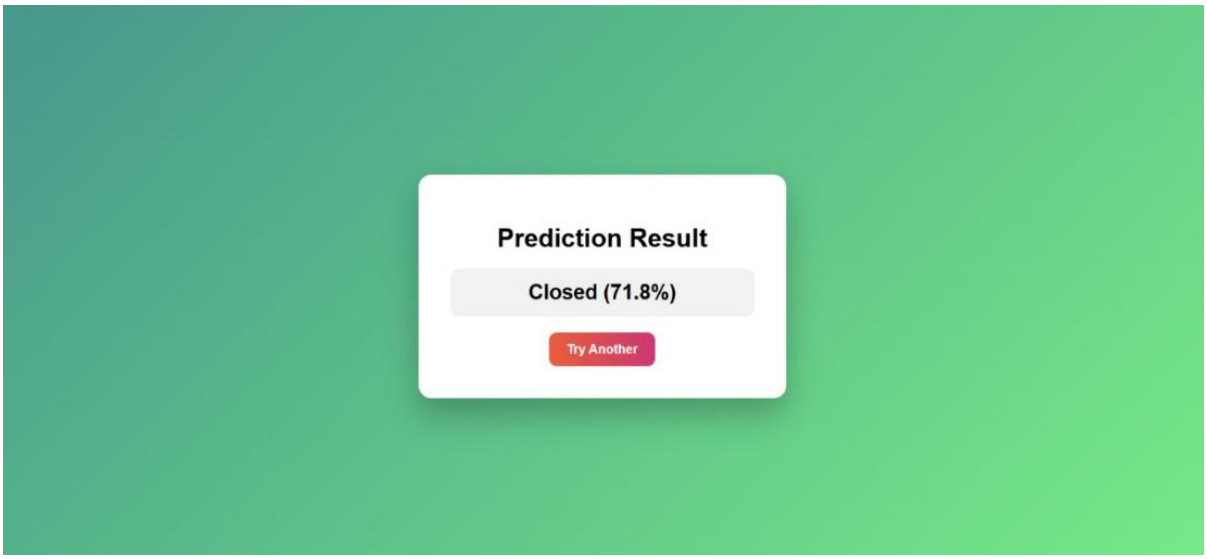
For this project create three HTML files namely

- index.html
- result.html

and save them in templates folder.
Let's see how our index.html page looks like:



Now when you click on predict button from top right corner you will get redirected to predict.html
Lets look how our result.html file looks like:



Activity 2: Build Python code:
Import the libraries

```
from flask import Flask, render_template, request
import numpy as np
import pickle
```

Load the saved model. Importing flask module in the project is mandatory. An object of Flask class is our WSGI application. Flask constructor takes the name of the current module (__name__) as argument.
Render HTML page:

```
@app.route("/")
def index():
    return render_template("index.html")
```

Here we will be using declared constructor to route to the HTML page which we have created earlier.
In the above example, '/' URL is bound with home.html function. Hence, when the home page of the web server is opened in browser, the html page will be rendered.
Whenever you enter the values from the html page the values can be retrieved using POST Method.
Retrieves the value from UI:

```python
from flask import Flask, render_template, request
import numpy as np
import joblib

app = Flask(__name__)

model = joblib.load("model.pkl")
scaler = joblib.load("scaler.pkl")

@app.route("/")
def home():
    return render_template("index.html")

@app.route("/predict", methods=["POST"])
def predict():

    values = [
        float(request.form["age_first_funding_year"]),
        float(request.form["age_last_funding_year"]),
        float(request.form["age_first_milestone_year"]),
        float(request.form["age_last_milestone_year"]),
        float(request.form["relationships"]),
        float(request.form["funding_rounds"]),
        float(request.form["funding_total_usd"]),
        float(request.form["milestones"]),
        float(request.form["avg_participants"])
    ]

    data = np.array([values])
    data = scaler.transform(data)

    proba = model.predict_proba(data)[0]
    print("Probabilities:", proba)

    pred = model.predict(data)[0]

    if pred == "acquired":
```

Here we are routing our app to predict() function. This function retrieves all the values from the HTML page using Post request. That is stored in an array. This array is passed to the model.predict() function. This function returns the prediction. And this prediction value will rendered to the text that we have mentioned in the submit.html page earlier.

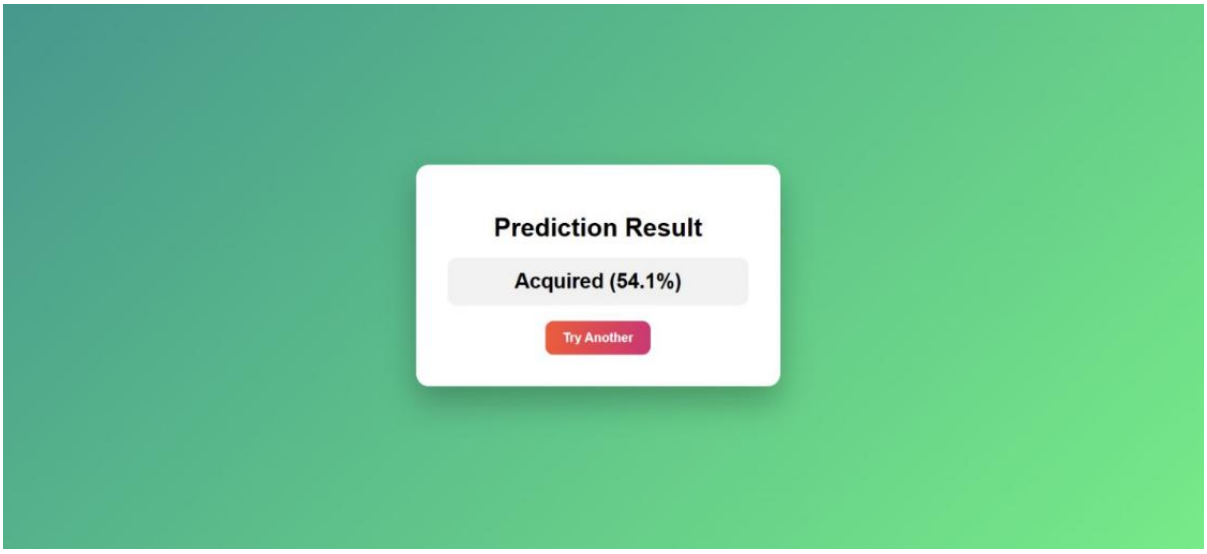
Main Function:

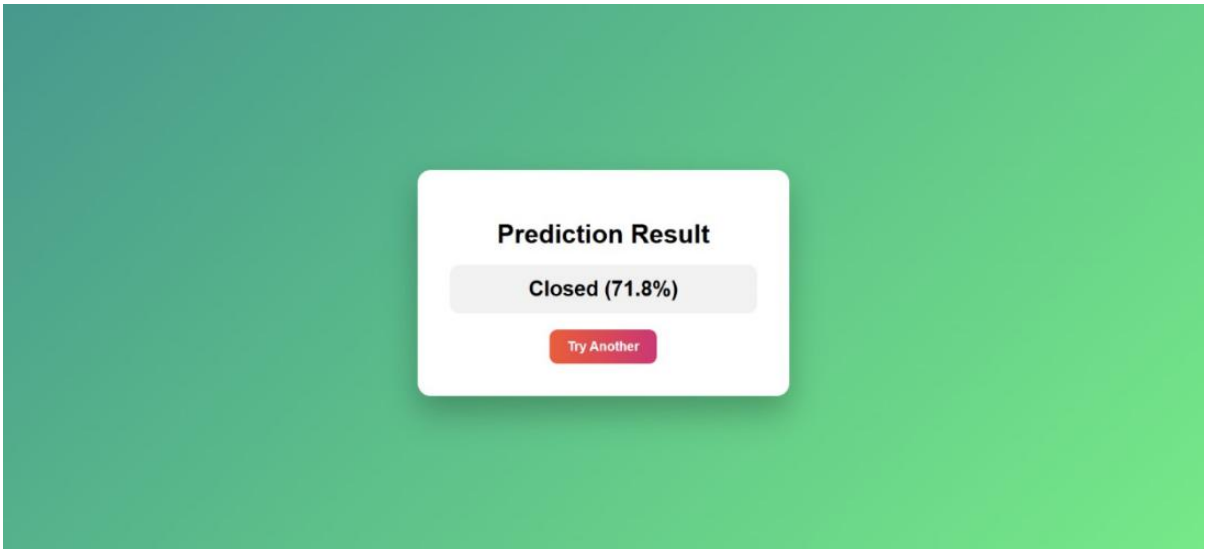
```python
if __name__ == "__main__":
    app.run(debug=True)
```

Activity 3: Run the application

- Open anaconda prompt from the start menu
- Navigate to the folder where your python script is.
- Now type “python app.py” command
- Navigate to the localhost where you can view your web page.
- Click on the predict button from top left corner, enter the inputs, click on the submit button, and see the result/prediction on the web.

```
(venv) PS C:\Users\admin\Desktop\wind energy prediction> python app.py
>>
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 106-532-199
```


Prediction Result
Acquired (54.1%)
Try Another


Prediction Result
Closed (71.8%)
Try Another